Smart Streetlight Energy Consumption Prediction

Objective

To predict streetlight electricity consumption using Machine Learning and Time Series Forecasting.

Models Used

- Linear Regression
- Random Forest
- Decision Tree
- Gradient Boosting
- Time Series Forecast Model

In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

Dataset Description

Dataset contains smart streetlight parameters:

- Zone Information
- Weather Conditions
- Traffic Density
- Pedestrian Density
- Light Intensity
- Maintenance Score

Target Variable: Energy Consumption (kWh)

In [7]:
```
df = pd.read_csv(r"C:\ML Project\StreetLight_Project\streetlight_energy_dataset.csv
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (2000, 14)
```

Out[7]:

	zone_id	area_type	hour	day_of_week	month	temperature	humidity	weather_condit
0	4	2	21	6	2	17.242523	42.884670	
1	5	2	20	5	10	39.038593	51.358103	
2	3	1	7	1	6	35.181541	47.046249	
3	5	0	16	0	12	26.490819	40.565357	
4	5	1	15	4	12	20.590064	74.599975	

In [8]:

```
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   zone_id                 2000 non-null   int64
 1   area_type               2000 non-null   int64
 2   hour                    2000 non-null   int64
 3   day_of_week             2000 non-null   int64
 4   month                   2000 non-null   int64
 5   temperature             2000 non-null   float64
 6   humidity                2000 non-null   float64
 7   weather_condition       2000 non-null   int64
 8   traffic_density         2000 non-null   float64
 9   pedestrian_density      2000 non-null   float64
 10  light_intensity         2000 non-null   float64
 11  lamp_age                2000 non-null   float64
 12  maintenance_score       2000 non-null   float64
 13  energy_consumption_kwh  2000 non-null   float64
dtypes: float64(8), int64(6)
memory usage: 218.9 KB
None

Missing Values:
zone_id                   0
area_type                 0
hour                      0
day_of_week               0
month                     0
temperature               0
humidity                  0
weather_condition         0
traffic_density           0
pedestrian_density        0
light_intensity           0
lamp_age                  0
maintenance_score         0
energy_consumption_kwh    0
dtype: int64
```

Feature Selection

Energy Consumption is target variable. All other features are input variables.

In [9]:
```
X = df.drop("energy_consumption_kwh", axis=1)
y = df["energy_consumption_kwh"]
```

In [10]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Machine Learning Model Training

Multiple models are trained and compared. Best model selected based on R2 Score.

In [11]:
```
models = {
    "Multiple Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

results = []
best_model = None
best_model_name = ""
best_r2 = -999

n = len(X_test)
p = X_test.shape[1]

for name, model in models.items():

    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    r2 = r2_score(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, pred)

    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    print(f"\n{name}")
    print("R2:", r2)
    print("Adjusted R2:", adj_r2)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("MAE:", mae)

    results.append([name, r2, adj_r2, mse, rmse, mae])

    if r2 > best_r2:
        best_r2 = r2
        best_model = model
        best_model_name = name

results_df = pd.DataFrame(results, columns=[
    "Model", "R2", "Adjusted R2", "MSE", "RMSE", "MAE"
])

print("\n===== MODEL COMPARISON TABLE =====")
results_df.sort_values(by="R2", ascending=False)
```

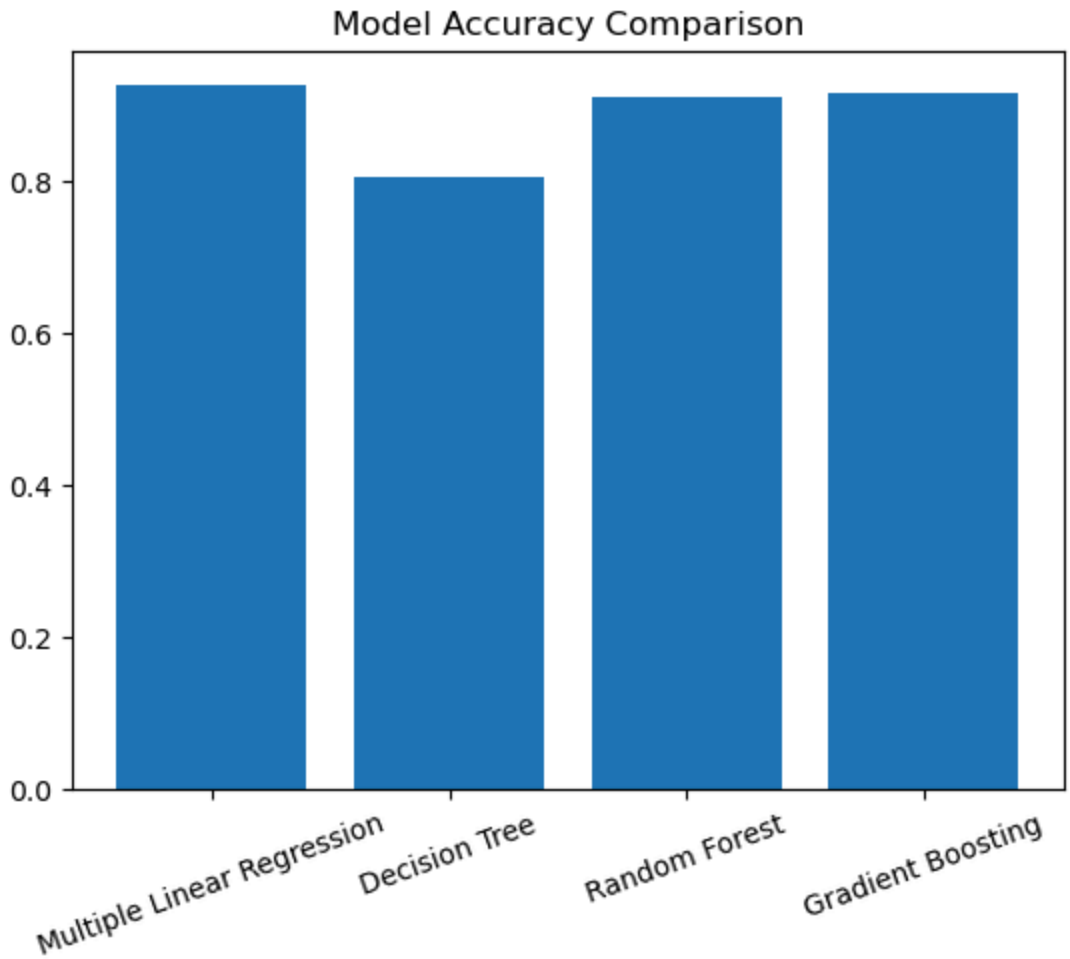
```
Multiple Linear Regression
R2: 0.9259931566078292
Adjusted R2: 0.9235006981516162
MSE: 8.68178256306029
RMSE: 2.946486477664591
MAE: 2.5310581407689914

Decision Tree
R2: 0.8065912864654804
Adjusted R2: 0.8000775215018826
MSE: 22.68888010545186
RMSE: 4.763284592112029
MAE: 3.867052320710966

Random Forest
R2: 0.9105665294186762
Adjusted R2: 0.9075545213421031
MSE: 10.491488487523334
RMSE: 3.2390567280495928
MAE: 2.725325593566388

Gradient Boosting
R2: 0.9177793863197509
Adjusted R2: 0.9150102982942503
MSE: 9.645344368907626
RMSE: 3.1056954726610955
MAE: 2.6077812118196038

===== MODEL COMPARISON TABLE =====
```

Out[11]:

	Model	R2	Adjusted R2	MSE	RMSE	MAE
0	Multiple Linear Regression	0.925993	0.923501	8.681783	2.946486	2.531058
3	Gradient Boosting	0.917779	0.915010	9.645344	3.105695	2.607781
2	Random Forest	0.910567	0.907555	10.491488	3.239057	2.725326
1	Decision Tree	0.806591	0.800078	22.688880	4.763285	3.867052

In [12]:

```
plt.figure()
plt.bar(results_df["Model"], results_df["R2"])
plt.title("Model Accuracy Comparison")
plt.xticks(rotation=20)
plt.show()
```



Feature Importance Analysis

Shows which features influence energy consumption most.

In [13]:
```
print("Best Model Selected:", best_model_name)

if hasattr(best_model, "feature_importances_"):

    fi = pd.DataFrame({
        "Feature": X.columns,
        "Importance": best_model.feature_importances_
    })

    display(fi.sort_values(by="Importance", ascending=False))

elif hasattr(best_model, "coef_"):

    fi = pd.DataFrame({
        "Feature": X.columns,
        "Coefficient": best_model.coef_
    })

    display(fi.sort_values(by="Coefficient", ascending=False))
```

```
Best Model Selected: Multiple Linear Regression
```

	Feature	Coefficient
10	light_intensity	0.496595
8	traffic_density	0.199972
9	pedestrian_density	0.098898
3	day_of_week	0.020383
4	month	0.017182
12	maintenance_score	0.006277
2	hour	0.005654
6	humidity	-0.003613
5	temperature	-0.008490
1	area_type	-0.017480
0	zone_id	-0.029963
7	weather_condition	-0.036915
11	lamp_age	-0.037236

In [14]:
```
joblib.dump(best_model, r"C:\ML Project\StreetLight_Project\energy_model.pkl")
print("Best Model Saved Successfully")
```

```
Best Model Saved Successfully
```

Time Series Forecasting

Time series model predicts future energy usage based on past energy consumption patterns.

In [15]:
```
df["datetime"] = pd.date_range(
    start="2024-01-01",
    periods=len(df),
    freq="h"
)

df = df.sort_values("datetime")

ts_df = df[["datetime", "energy_consumption_kwh"]].copy()
ts_df.set_index("datetime", inplace=True)

ts_df["lag_1"] = ts_df["energy_consumption_kwh"].shift(1)
ts_df["lag_2"] = ts_df["energy_consumption_kwh"].shift(2)
ts_df["lag_3"] = ts_df["energy_consumption_kwh"].shift(3)
ts_df["rolling_mean_3"] = ts_df["energy_consumption_kwh"].rolling(3).mean()

ts_df.dropna(inplace=True)
```

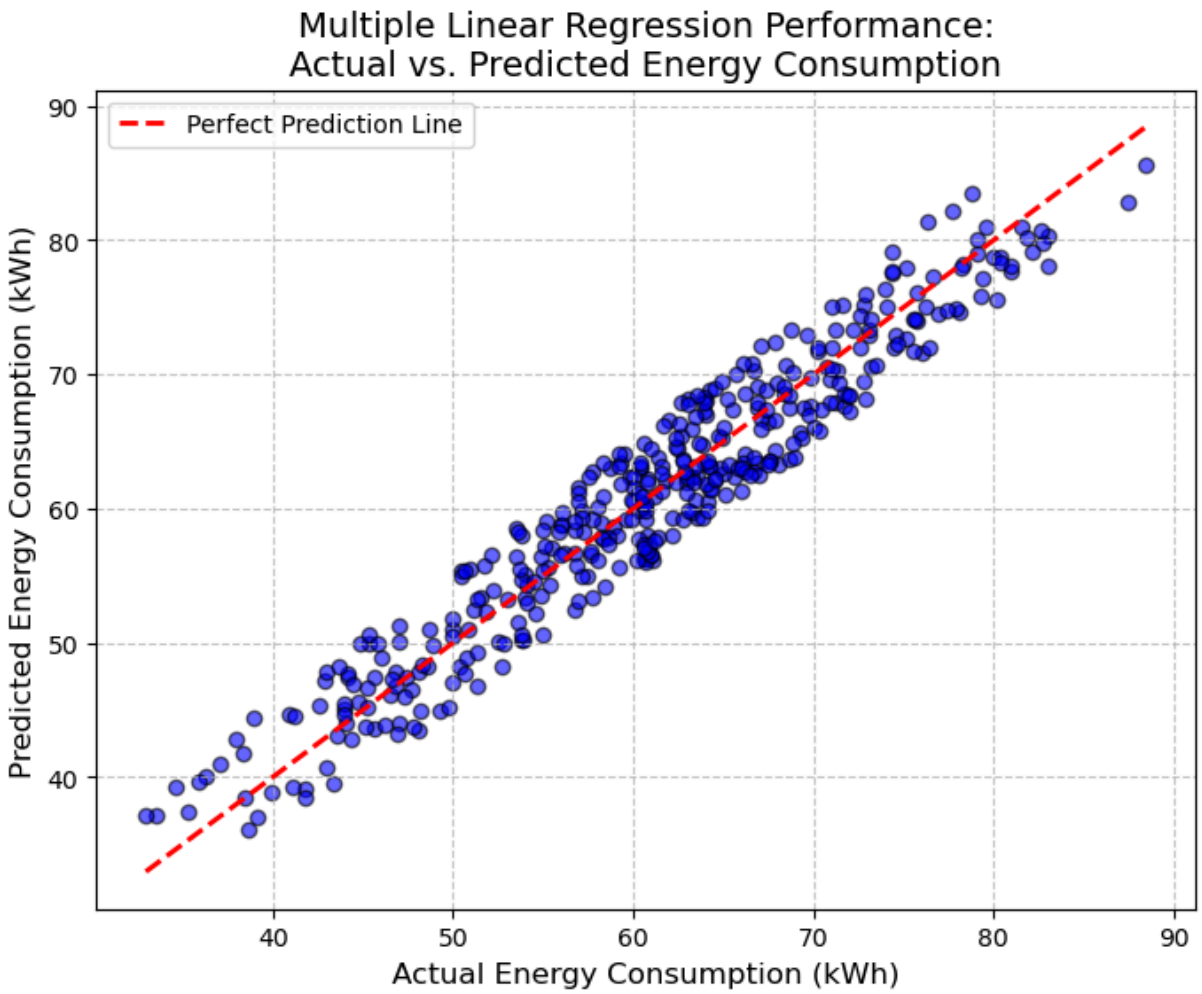
```
X_ts = ts_df[["lag_1", "lag_2", "lag_3", "rolling_mean_3"]]
y_ts = ts_df["energy_consumption_kwh"]

split = int(len(ts_df) * 0.8)

X_train_ts = X_ts[:split]
X_test_ts = X_ts[split:]
y_train_ts = y_ts[:split]
y_test_ts = y_ts[split:]

ts_model = RandomForestRegressor(n_estimators=200, random_state=42)
ts_model.fit(X_train_ts, y_train_ts)

ts_pred = ts_model.predict(X_test_ts)

ts_r2 = r2_score(y_test_ts, ts_pred)
ts_mse = mean_squared_error(y_test_ts, ts_pred)
ts_rmse = np.sqrt(ts_mse)
ts_mae = mean_absolute_error(y_test_ts, ts_pred)

print("\nTime Series Results")
print("R2:", ts_r2)
print("MSE:", ts_mse)
print("RMSE:", ts_rmse)
print("MAE:", ts_mae)
```

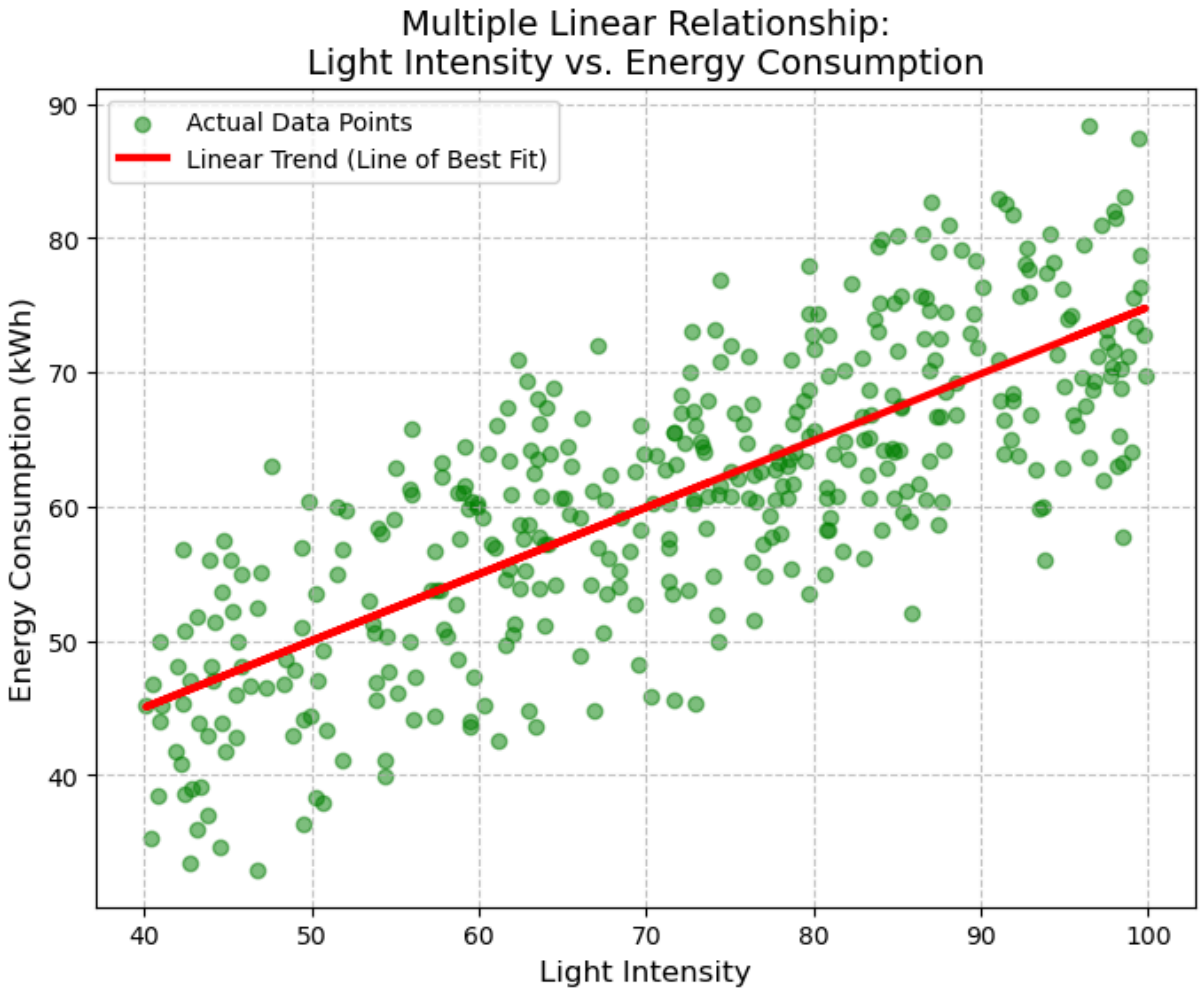
```
Time Series Results
R2: 0.9474387459041536
MSE: 6.7182626229839775
RMSE: 2.591961153833903
MAE: 1.87758683553092
```

In [17]:

```
import matplotlib.pyplot as plt
import numpy as np

# Make sure 'pred' contains the predictions from the Linear Regression model

lin_reg_model = models["Multiple Linear Regression"]
lin_reg_pred = lin_reg_model.predict(X_test)

# ==========================================
# GRAPH 1: Actual vs. Predicted Energy
# ==========================================
plt.figure(figsize=(8, 6))
plt.scatter(y_test, lin_reg_pred, alpha=0.6, color='blue', edgecolors='k')

# Drawing the perfect 45-degree prediction line
min_val = min(y_test.min(), lin_reg_pred.min())
max_val = max(y_test.max(), lin_reg_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], color='red', linewidth=2, linestyl

plt.title('Multiple Linear Regression Performance:\nActual vs. Predicted Energy Con
plt.xlabel('Actual Energy Consumption (kWh)', fontsize=12)
plt.ylabel('Predicted Energy Consumption (kWh)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.show()

# ==========================================
# GRAPH 2: Strongest Feature vs. Target
# ==========================================
plt.figure(figsize=(8, 6))
plt.scatter(X_test['light_intensity'], y_test, alpha=0.5, color='green', label='Act

# Calculating the line of best fit for just this feature
m, b = np.polyfit(X_test['light_intensity'], y_test, 1)
plt.plot(X_test['light_intensity'], m * X_test['light_intensity'] + b, color='red',

plt.title('Multiple Linear Relationship:\nLight Intensity vs. Energy Consumption',
plt.xlabel('Light Intensity', fontsize=12)
plt.ylabel('Energy Consumption (kWh)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```





Conclusion

• Successfully developed a Smart Streetlight Energy Prediction System

• Implemented Multiple Machine Learning Models:

- Linear Regression
- Random Forest
- Decision Tree
- Gradient Boosting

• Compared models using:

- R^2
- Adjusted R^2
- MSE
- RMSE
- MAE

• Automatically selected the Best Performing Model

- Implemented Time Series Forecasting for future energy prediction
- System can help optimize electricity usage in smart city environments
- Project ready for real-world deployment

In [ ]:

```

```

In [ ]:

```

```